

CareerLens AI

AI-Powered Interview Preparation & Job Search Assistant

1. Overview

CareerLens AI is a full-stack web application that helps candidates prepare for interviews and find relevant jobs. It analyzes a user's resume or self-description against a target job posting, generates a structured interview report, scrapes live job boards for matching openings, and can draft and send cold emails with ATS-safe resume attachments.

Core Features

- **AI Interview Reports** — match score, technical & behavioral questions with model answers, skill-gap analysis, and a day-wise preparation plan.
- **Multi-Platform Job Search** — live-scrapes jobs from LinkedIn, Wellfound, and Indeed; scores and ranks them against the candidate's profile.
- **Cold Email Assistant** — drafts a personalized email to a founder/hiring manager and sends it via SMTP (or Ethereum for testing).
- **ATS-Safe Resume Generation** — builds a single-column, machine-parseable resume PDF tailored to a specific posting.
- **JWT Authentication** — register/login with cookie-based sessions and a token blacklist for logout.

2. Architecture

The project is a monorepo with two independent Node apps:

```
GenAI/
├── Backend/           Express REST API (CommonJS)
│   ├── server.js      Entry point (port 3000)
│   └── src/
│       ├── app.js      Express app, middleware, routes
│       ├── db/          MongoDB connection (Mongoose)
│       ├── models/      User, InterviewReport, blacklist
│       ├── routes/       /api/auth, /api/report, /api/jobs
│       ├── controllers/  Request handlers
│       ├── middleware/   JWT auth, multer uploads
│       └── services/     AI, scraping, PDF, email logic
└── Frontend/          React 19 + Vite SPA (ESM)
    └── src/
        ├── main.jsx      Entry point
        ├── App.jsx        Providers + router
        ├── app.routes.jsx Client routing
        └── features/
            ├── auth/      Login, Register, Auth context
            ├── ai/         Interview report pages & hooks
            └── jobs/       Matched jobs & cold email UI
```

Request Flow

Frontend (React + Vite, port 5173)

↓ axios with credentials

Backend (Express, port 3000)

↓ authMiddleware (JWT cookie check + blacklist)

Controllers — parse request, call services

↓

Services — AI calls, scraping, PDF, email

↓

MongoDB (Mongoose ODM)

3. Tech Stack

Layer	Technology
Runtime	Node.js
Backend Framework	Express 5
Frontend Framework	React 19
Build Tool	Vite 8
Language	JavaScript (CommonJS backend, ESM frontend)
Database	MongoDB (Mongoose 9 ODM)
AI	Google Generative AI (Gemini 3.5 Flash)
Scraping	Playwright, Puppeteer, Cheerio
PDF	Puppeteer (HTML to PDF), pdf-parse
Email	Nodemailer
Auth	JWT (cookie), bcrypt
Validation	Zod
Styling	SCSS, Tailwind CSS 4
Routing	react-router-dom v7 (createBrowserRouter)

4. Backend — Key Modules

4.1 Authentication (`authController.js` + `auth.middleware.js`)

- Register/login with bcrypt-hashed passwords.
- JWT signed and stored in an httpOnly cookie named `token`.
- `authMiddleware` verifies the JWT and checks the `blacklist` collection for logged-out tokens.
- Protected routes mount `authMiddleware` before the controller.

4.2 Interview Reports (`reportController.js` + `ai.service.js`)

- User uploads a resume PDF (via `multer`, in-memory, 3 MB limit) and pastes a job description.
- Resume text is extracted with `pdf-parse`.
- Gemini is called with a structured JSON schema: `matchScore`, `technicalQuestions`, `behavioralQuestions`, `skillGaps`, `preparationPlan`, `title`.
- The response is validated with Zod and saved to MongoDB.
- Resume PDF can be generated from the report data (ATS-safe, single-column).

4.3 Job Search (`jobController.js` + `job.service.js` + `scraper.service.js`)

- **Profile extraction** — Gemini extracts the target role and skills from the resume. Falls back to local regex + skill detection when the API is unavailable.
- **Scraping** — Playwright launches headless Chromium (with Puppeteer fallback). Scrapes LinkedIn (guest API), Wellfound (role pages), and Indeed. Each scraper handles bot detection, rate limiting, and deduplication.
- **Enrichment** — LinkedIn and Indeed detail pages are fetched to add full job descriptions and employment type.
- **Scoring** — Keyword overlap scoring ranks postings against the candidate's skills. Sorted by score, then freshness.
- **Diagnostics** — blocked platforms and errors are reported to the frontend so partial results are not mistaken for "no matches".

4.4 Cold Email (`job.service.js` + `email.service.js`)

- Gemini drafts a personalized cold email (subject + body) using the candidate's profile and the job posting.
- Email is sent via Nodemailer. Supports Gmail, custom SMTP, or Ethereal test transport (preview URL, no delivery).
- Resume attachment options: user's own PDF, ATS-generated PDF, or no attachment.

4.5 ATS Resume (`atsResume.service.js`)

- Gemini extracts structured facts (name, skills, experience, projects, education) from the profile.
- Renders a single-column HTML resume with standard headings and real `<ul>/<li>` bullets.
- Converts to PDF via Puppeteer.
- Reads the PDF back with `pdf-parse` to verify the ATS can extract text (name, skills recoverable).

5. Frontend — Feature Breakdown

Feature	Pages	Context / Hooks	API Service
Auth	Login, Register	AuthContext (user, loading)	auth.api.js
AI Interview	Home (input), Interview (report)	InterviewContext, useInterview	interview.api.js
Jobs	Jobs (results + cold email modal)	—	job.api.js

Routing (react-router-dom v7)

Path	Component	Auth
/login	Login	No
/register	Register	No
/	Home	Yes (Protected)
/report/:interviewId	Interview	Yes (Protected)
/jobs	Jobs	Yes (Protected)

State Management

- `AuthProvider` — holds current user and loading state across the app.
- `InterviewProvider` — holds the current report, list of reports, and loading state.
- Custom hook `useInterview` — wraps API calls (generate, fetch by ID, list all, download PDF).
- Job results are passed via `react-router` location state and persisted in `sessionStorage`.

Styling

- SCSS with BEM-ish naming (`.q-card__header` , `.score--high`).
- Tailwind CSS 4 via `@tailwindcss/vite` plugin.

6. API Reference

Auth — `/api/auth`

Method	Path	Auth	Description
POST	/register	No	Register a new user
POST	/login	No	Login (sets JWT cookie)
GET	/logout	No	Logout (blacklists token)
GET	/get-me	Yes	Get current user

Reports — `/api/report`

Method	Path	Auth	Description
POST	/	Yes	Generate interview report (multipart)
GET	/	Yes	List user's reports
GET	/:reportId	Yes	Get one report
POST	/resume-pdf/:reportId	Yes	Generate resume PDF

Jobs — `/api/jobs`

Method	Path	Auth	Description
POST	/match	Yes	Find matching jobs (multipart)
POST	/cold-email	Yes	Generate cold email
POST	/send-email	Yes	Send cold email + optional resume
POST	/ats-resume	Yes	Build/preview ATS resume PDF

7. Environment Setup

Required (Backend/.env)

```
MONGO_URI=mongodb://localhost:27017/careerlens
GOOGLE_GENAI_API_KEY=your_gemini_api_key
JWT=your_jwt_secret
```

Optional (email)

```
EMAIL_USER=your_email
EMAIL_PASS=your_app_password
EMAIL_SERVICE=gmail
```

If omitted, mail falls back to Ethereum test transport (returns a preview URL, does not deliver).

Running

```
# Terminal 1 – Backend
cd Backend
npm install
npm run dev

# Terminal 2 – Frontend
cd Frontend
npm install
npm run dev
```

Open `http://localhost:5173` .

8. Known Limitations & Notes

- **Scrapers depend on live sites** — job boards change markup and rate-limit bots. The backend reports diagnostics (which platforms were blocked) instead of silently returning empty results.
- **Naukri removed** — Naukri.com aggressively blocks headless browsers (403 on every request). It has been removed from the scraper provider list.
- **AI scoring removed for speed** — Sending 80+ postings to Gemini for scoring caused hangs on 503/429 responses. Keyword overlap scoring is used instead (instant, no API dependency).
- **Internships removed** — Only full-time job postings are searched and displayed.
- **Gemini availability** — Profile extraction falls back to local regex + skill detection when the model is unavailable.
- **Email** — Without SMTP credentials, mail goes through Ethereal (test only).
- **Dev CORS** — The backend only allows origin `http://localhost:5173`.
- **No tests** — `npm test` is not configured. Verify changes by running the dev servers.